EagleEyes Parking

Group 3

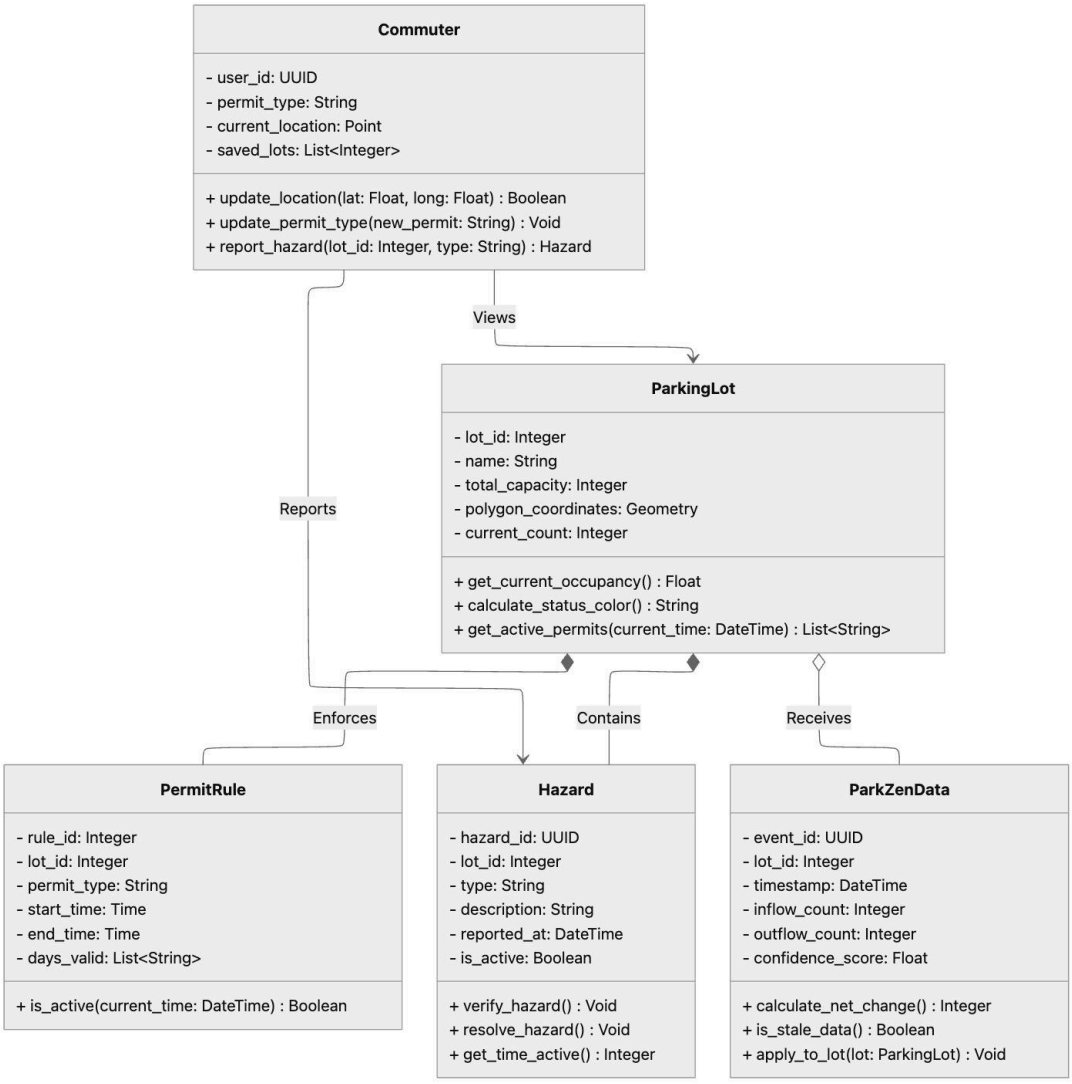
Design Document 1

Team lead
Nhan Vu
Team members
Thuan Nguyen
Alperen Dalkir
Luis Lozano
Aaditya Yadav

Sponsor: Jim Coffey (UNT Transportation Services)

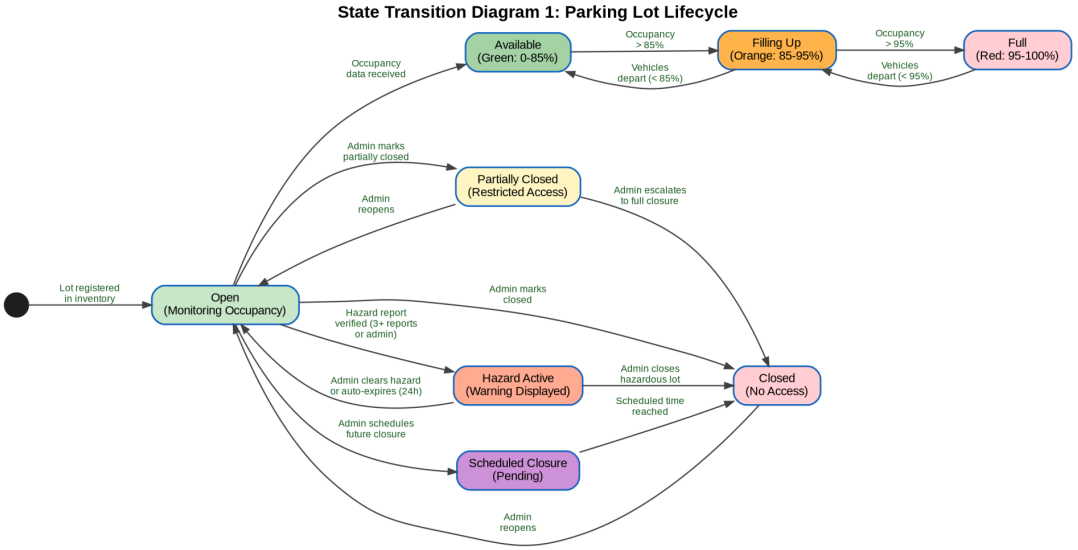
CSCE4901.502-Capstone1(Spring 2026)

1. Class Diagram

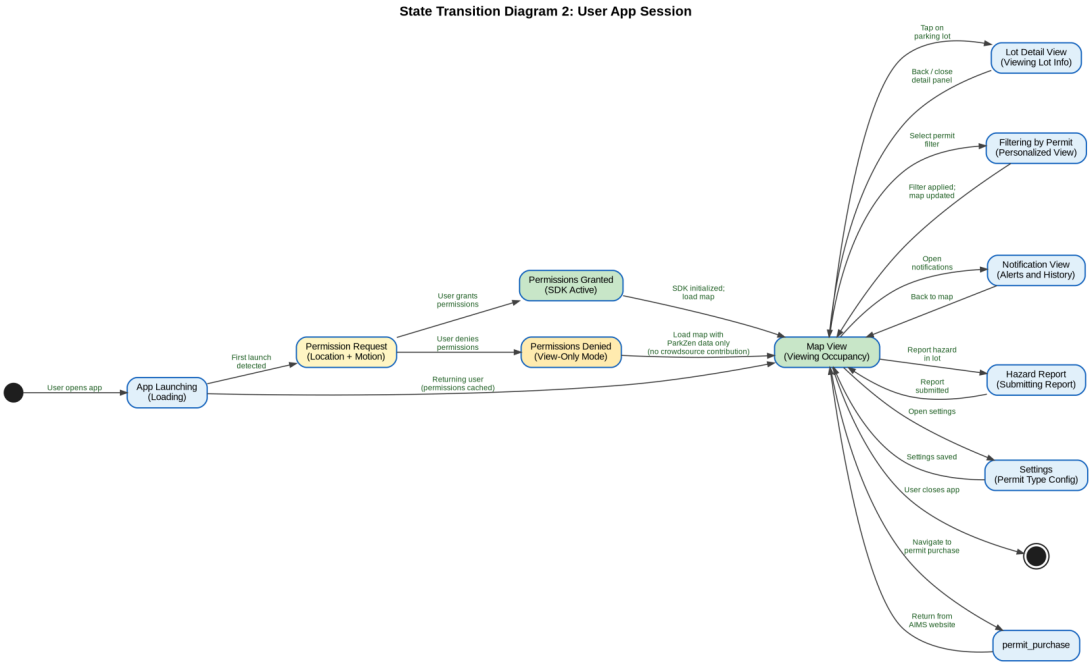


The class diagram illustrates the main structural components of the system, including the classes Commuter, PermitRule, Hazard, and ParkZenData. Each class contains its own set of variables, which represent the data it stores, and methods (functions), which define the behaviors or operations that can be performed on that data. Together, these variables and methods enable each class to fulfill its specific role within the system and interact with other classes effectively.

2. State Transition Diagrams

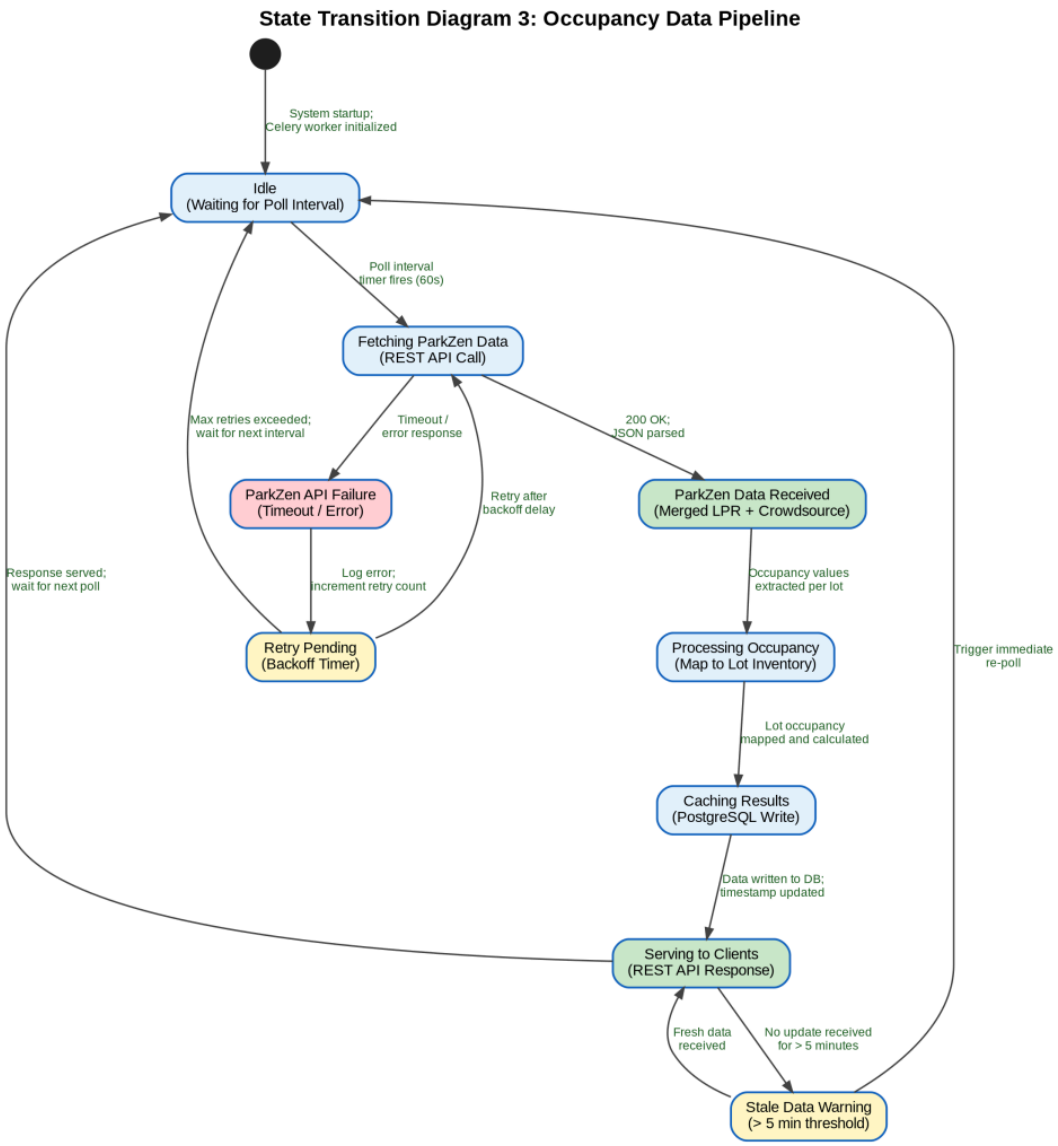


This diagram presents the cycles that a parking lot passes through since it is registered in the system. It deals with the 3-tier occupancy flow (Green 0-85, Orange 85-95, Red 95-100), and the actions of the administration, such as partial and scheduled closures and hazard management. In the development process, it informs the backend of the way lot status is to be modeled in the database and what API responses the frontend must have in order to display lot colors and closure state on the map.

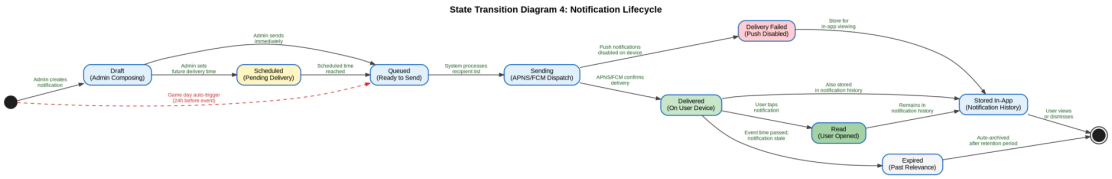


A flow chart visually represents the experience of the user, starting with opening the application, and the permission request process, then into the core map screen, and all the

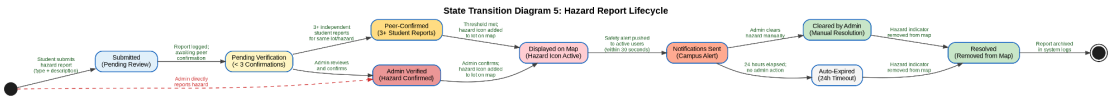
screens that the user can navigate to, including lot details, permit filtering and notification, and settings. It can be used as a guide to frontend routing and screen navigation code, and make the UI/UX team members and frontend developers aware of all possible paths the user can take and that there are no dead-end states in the application.



This diagram describes the form of the backend Celery worker, whereby the worker polls the ParkZen API every minute, gets the response, which already includes merged LPR and crowdsource data, caches the response in PostgreSQL, and makes the response available to clients. It also includes error processing of retry/backoff logic and stale data warnings. This guides directly the way the Fast API ends and the background task services should be created during development.

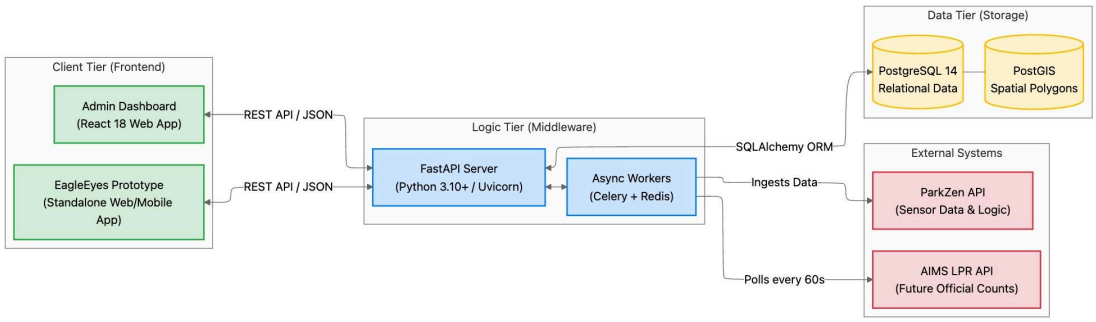


This flow chart follows the path of an announcement (created by an administrator or automatic during game days) through scheduling, sending through APNS/FCM, delivery, and archiving in the in-app history. It takes into consideration untabulated deliveries and expiration. This will assist developers to use the notification service, set the notification database schema and create the notification management interface of the admin dashboard.



This flow chart demonstrates the process of a hazard report being submitted by students, peer-reviewed (10 or more confirmations) or just administrationally reviewed, and appearing on the map and causing campus alerts within 30 seconds. It has the auto-expiration of 24 hours and clearance by admins. This directs the process of implementation of the hazard reporting API endpoints, verification logic and real-time alert system.

3. System Design Architecture Diagram



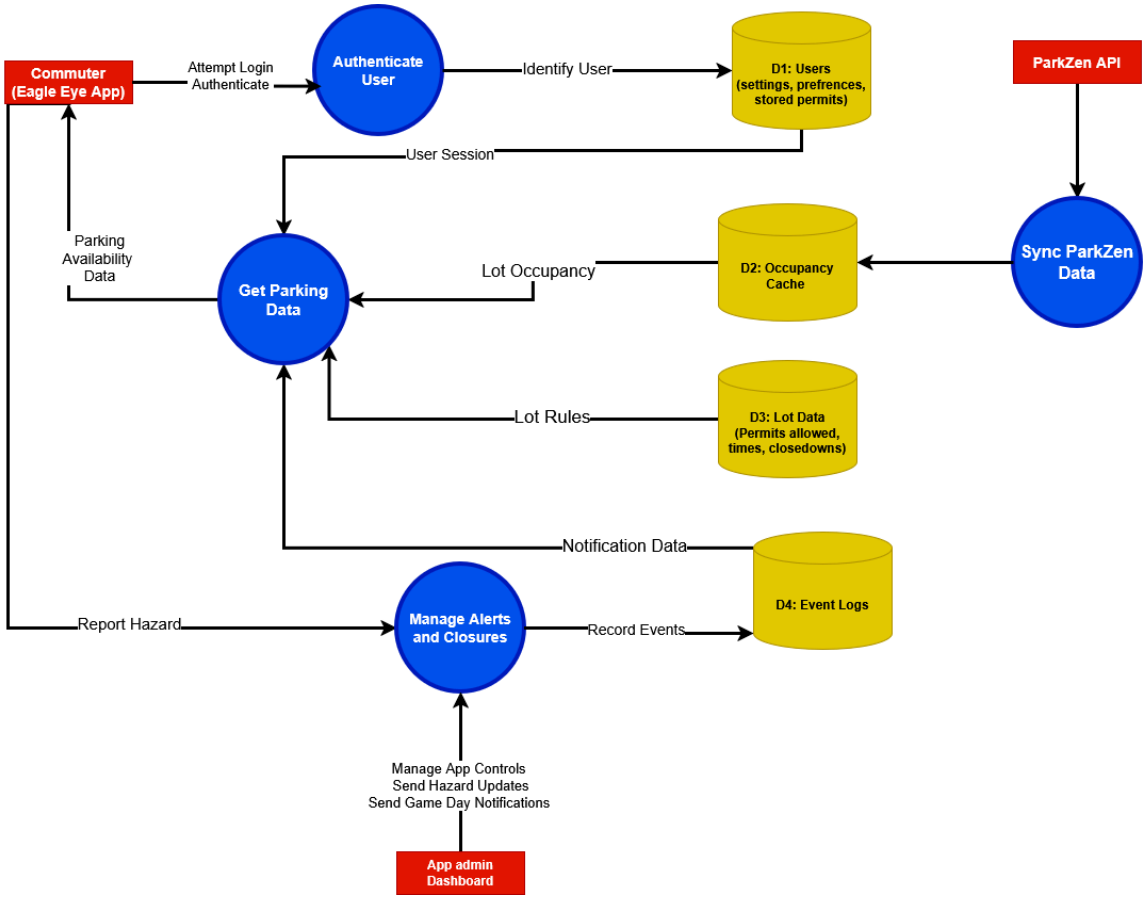
The System Design Architecture Diagram shows how the system is structured and how each tier interacts. The project is divided into four main tiers: client, logic, data, and external systems.

The client tier includes user interfaces, such as mobile or web apps. The logic tier handles backend processing, including APIs, authentication, and business rules. The data tier stores information such as user data, permits, and hazard reports in databases (e.g.,

MySQL). External systems include third-party services such as Google Maps APIs, payment systems, or notification services.

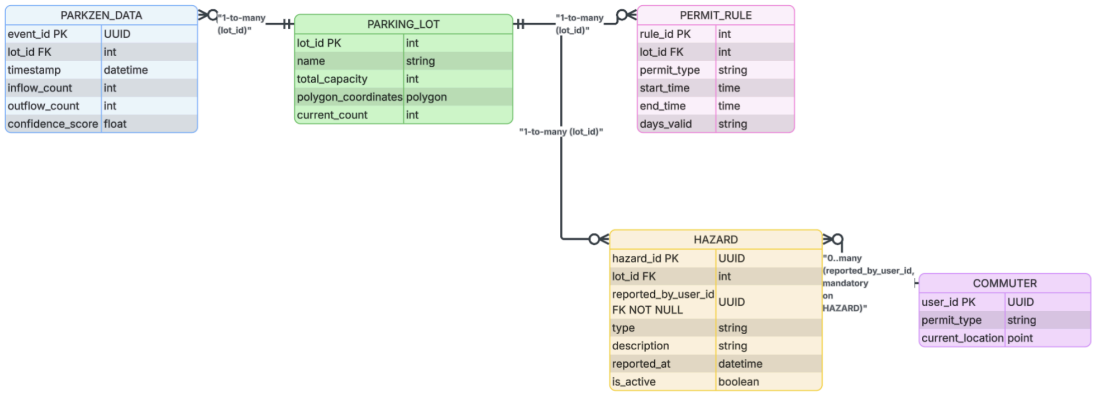
Arrows in the diagram represent how data flows between tiers: the client sends requests to the backend, which processes them, interacts with the database or external services, and returns a response.

4. Data Flow Diagram (DFD)



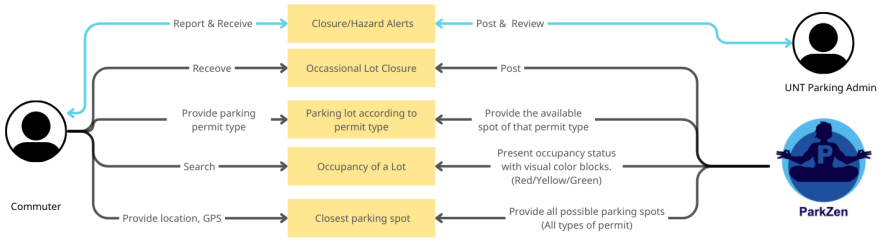
The dataflow diagram shows how data will be exchanged within the app and with users, including databases and external data sources such as our API call to parkzen. The data flow goes from the user, who authenticates the session. The session then fetches the profile, and the user's request for lot occupancy fetches the most recent cached data from ParkZen. After all the relevant data is collected, it sends the user the occupancy data, which appears on the app's frontend.

5. Entity-Relationship Diagram (ERD)



The Entity-Relation Diagram illustrates the system's overall structure by defining the main entities, their attributes, and the relationships between them. Each entity represents a key component of the system, while the relationships show how data is connected and shared across different parts. The diagram also includes cardinality to indicate how entities interact, such as one-to-one or one-to-many relationships. This design helps ensure data is organized, consistent, and easy to manage, providing a clear foundation for implementing and maintaining the database.

6. Use Case Diagram



This flowchart shows how commuters, the UNT Parking Administration, and ParkZen interact to manage parking. It outlines how alerts are shared, how lot closures are communicated, how parking availability is determined by permit type, how lot occupancy is displayed with color indicators, and how the system provides the closest available parking spots using GPS.

NOTE: Below is the link to a folder with all of the diagrams in case the image is not viewable.

https://drive.google.com/drive/folders/1ZFgC-lBNClzu-n7O3vaOJfW3_oHIQ2ao?usp=drive_link

t